UN MANUALE TECNICO DI BM

Catalogo delle tecniche grafiche

Ogni tecnica 2D e 3D di bm su CPU, V3D (Pi Zero W) e Mali-G52 (RGB30): che cos'è, cosa è costruito, quanto costa e cosa non è costruito.

Indice

Come leggere le tabelle **3**

3D sulla CPU (rasterizzatore ARM) **4**

L'ARM su quattro core (RGB30) 5

3D sul V3D (Pi Zero W) **7**

3D sul Mali-G52 (RGB30) **10**

2D sulla CPU **12**

2D sulle GPU **14**

L'elenco OpenVG e VGU (vettoriale 2D) **16**

Cosa hanno le altre API e bm no **18**

Punti aperti **19**

Punti aperti **20**

LEGGIMI

Che cos'è. Un elenco, una tecnica alla volta, di ciò che bm fa per disegnare: rasterizzazione 3D, illuminazione, texture, sprite e mappe 2D, tabelle di luce e il lavoro che le GPU prendono. Ogni riga dice se la tecnica è **Fatta**, **Parziale**, **Non fatta** o **Scartata**, e perché quando non è fatta.

Come si legge. Tre macchine: l'**ARM** (Pi Zero W, un core; RGB30, quattro core), il **V3D** (VideoCore IV del Pi Zero W) e il **Mali** (Mali-G52 della RGB30). L'ultima colonna dà l'equivalente nelle API pubbliche (OpenGL ES, Vulkan, Direct3D, OpenVG, OpenCL, OpenVX, SDL2 e SDL3, Mesa) per il confronto.

Da dove vengono i numeri. Report del 3D Bench, 640x360, RGB565 sul Pi e XRGB sulla RGB30. Fonti: `docs/PERF-BUDGET.md`, `docs/progress/graphics.md`, `docs/progress/graphics-mali.md`, `docs/progress/graphics-vector.md`, `docs/progress/multicore.md`. Un numero marcato *non misurato* non è mai stato misurato. Niente è inventato.

Come leggere le tabelle

Parola	Significato
Fatta	Funziona nel kernel e ha un test.
Parziale	Funziona per alcuni casi. La riga dice quali.
Non fatta	Niente nel codice. La riga dà il motivo o dice che nessuno ha deciso.
Scartata	Provata o pianificata, poi lasciata fuori. La riga dà il motivo.

Le cifre di carico sono "a 60 fps": quante unità della cosa entrano in un frame di 16,6 ms. Il Pi Zero W è la macchina di riferimento. Un numero della RGB30 ha una nota propria.

3D sulla CPU (rasterizzatore ARM)

L'ARM trasforma, illumina, ritaglia e disegna i triangoli da solo. È l'unico percorso 3D della RGB30 prima del Mali e il ripiego di entrambe le GPU. Fonte: `src/bm/r3d.c` .

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Rasterizzatore di triangoli	Fatta	Funzioni di bordo in virgola fissa. Riempimento piatto o Gouraud (con dither), un pixel alla volta.	Pi: 70,8 sfere da 96 facce a 60 fps. RGB30: 177,3 (vecchio report, bm3d 6.8)	Raster della pipeline fissa GL; Vulkan `vkCmdDraw`
Buffer di profondità	Fatta	Z-buffer a 16 bit in RAM, azzerato con il DMA.	Parte delle cifre sopra	Test di profondità GL; allegato depth di Vulkan
Texture con prospettiva corretta	Fatta	Texel dallo sheet con la divisione corretta; cicli specializzati per caso.	Pi: eroe con texture 3,4 a 60 fps	GL `texture2D` ; sampler D3D
Ritaglio sul piano vicino	Fatta	L'ARM taglia i triangoli sul piano vicino.	Non misurato da solo	Volume di clip GL
Scarto delle facce posteriori	Fatta	È visibile il lato dove i vertici vanno in senso orario.	Non misurato da solo	`glCullFace`
Luce: sole, cielo e terra, speculare, rim, lampade puntuali	Fatta	Calcolata per vertice o per faccia dall'ARM con la VFP.	Parte delle cifre sopra	Luci fisse GL; luci D3D
Nebbia	Fatta	Mescola con un colore di nebbia secondo la profondità.	Non misurato da solo	`glFog` (GL 1.x)
Materiali per faccia	Fatta	Emissivo, lucido, screen-door (trasparenza a scacchiera), LOD.	Non misurato da solo	Stato materiale in D3D
Ombre planari	Fatta	Una copia appiattita della mesh su un piano.	Pi: eroe con ombra, l'ARM non arriva a 60 fps (24,7 ms con 1 eroe)	Ombre a proiezione (vecchia tecnica GL)
Skinning rigido	Fatta	Un osso per vertice; gruppi a due ossa sulla GPU.	Pi: eroe di 1536 triangoli con 16 ossa, 3,9 a 60 fps	Skinning GPU in un vertex shader

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Modo veloce (`r3d_fast=1`)	Fatta	Una matrice oggetto-camera, luce nello spazio del modello.	Dal 2,2 al 3,6 per cento di istruzioni ARM in meno per triangolo	-
Texture piatte (flag 128)	Fatta	Una faccia con texture è disegnata nel suo texel medio.	Non misurato da solo	-
Cel shading	Fatta	Luce divisa in 4 fasce. Opzione `dither3d` .	Costo rispetto alla luce del sole: x1,00 sull'ARM (Pi)	Rampa toon in un fragment shader
Contorno (guscio invertito)	Fatta	Disegna le facce posteriori più grandi, in un colore scuro (flag 512).	Da x1,34 a x1,50 dell'eroe illuminato sull'ARM (Pi)	Contorno a guscio (tecnica comune)
Punto, linea e billboard	Fatta	Sprite con profondità nella scena 3D.	Non misurato da solo	Point sprite
Impostori (`r3d_impostor`)	Parziale	Eroi lontani disegnati come immagine. Spento di default: una sessione Pi a 8 non ha dato guadagno.	Pi: 57,7 fps, GPU 7,8 ms a frame con valore 8	Impostori billboard
Particelle 3D	Fatta	Passo e disegno sull'ARM (`particles*`).	Non misurato sul Pi	Sistemi di particelle; compute shader
Sprite stacking	Fatta	Un modello fatto di fette (`stack` , `zstack` , `stackv`). Solo ARM.	Pi: 54,6 stack da 8 fette a 60 fps	- (tecnica di bm)
LOD per distanza	Fatta	Livello della mesh secondo la dimensione a schermo.	Non misurato da solo	Tassellazione / LOD
Shadow map	Non fatta	Nessuno ha deciso. Le ombre planari esistono.	-	Shadow mapping (GL, Vulkan, D3D)

L'ARM su quattro core (RGB30)

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Facce divise tra i core	Fatta	`offload_par` dà un blocco di facce a ogni core, da 96 facce, mesh piatte o con luce precalcolata e senza texture. Un core bloccato è segnato e il chiamante esegue il suo blocco.	Punteggio ARM del 3D Bench RGB30 466 -> 601	Elaborazione parallela dei vertici
Pixel in bande di righe	Fatta	Ogni core disegna le sue righe della pagina (`r3d_bands` , piatto, Gouraud, con texture). Stessi pixel di un core solo.	`heroes` 8,0 -> 15,8, `heroes_tex` 6,3 -> 12,7, `clip` 325 -> 692	Raster parallelo a tile
Costruzione dei draw per il Mali sui core	Fatta	`build_draws` in `mali.c` divide i draw tra i core in blocchi (da 32 draw). Chiave `mali_par` .	Giochi 451 -> 491, GPU 433 -> 513	Registrazione multi-thread dei comandi (Vulkan)
Core sul Pi Zero W	Non fatta	Il Pi Zero W ha un core solo. Un primo tentativo con 4 bande di tabelle ha perso la mailbox del firmware.	-	-
Luce per faccia sui core worker	Non fatta	Idea Q2 della coda Mali. Il ciclo è una funzione pura dell'intervallo di facce.	-	-

3D sul V3D (Pi Zero W)

Il V3D è il VideoCore IV (un renderer a tile). bm lo pilota senza Linux e senza la libreria 3D del firmware. Fonte: `src/gpu/gpu3d.c`, `v3d.c`, shader scritti in `tools/qpuasm.py`. Storia delle versioni: `docs/DRIVERS.md`.

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Binning e render a tile	Fatta	Il binner fa le liste dei tile; il renderer disegna tile per tile. Due job hardware a frame.	Grezzo: 2966 mila piccoli triangoli al secondo; 811 Mpixel al secondo (schermi interi, solo render)	GPU a tile (Mesa vc4)
Fragment shader sulle QPU	Fatta	Piatto, Gouraud, con texture, screen-door, cel.	`quad_flat`: 163,7 quarti di schermo a 60 fps (ARM 11,5)	Fragment shader GL
Vertex shader sulle QPU	Fatta	Trasformazione, luce, ossa, ombre; la GPU posiziona gli angoli (`gpu3d_vs=2`).	`heroes`: 66,3 a 60 fps (solo GPU 6,3, ARM 3,9); `spheres` 782,9 (ARM 70,8)	Vertex shader GL
Coordinate shader e clip vicino	Fatta	Uno shader separato dà le coordinate di clip al binner. La GPU ritaglia sul piano vicino e sul guard band.	Parte delle cifre dei vertex	Passo di binning Vulkan
Mesh indicizzate (16 bit)	Fatta	I vertici condivisi sono preparati una volta. Mesh fino a 65535 vertici e facce.	`big`: 9,2 modelli da 10080 triangoli	`glDrawElements`; `vkCmdDrawIndexed`
Profondità e early z	Fatta	Profondità a 24 bit nel tile buffer. L'early z è spento dopo uno `zclear()` e con MSAA.	Non misurato da solo	Test di profondità; early-Z
MSAA 4x	Fatta	Opzione `gpu3d_aa`. È l'unico antialiasing di bm.	Non misurato nelle schede usate qui	`GL_SAMPLES`; MSAA Vulkan
Coda dei job (1 job)	Fatta	Il job GPU parte a fine frame. Il prossimo `_update` gira nel frattempo. Attiva di default sul Pi.	Test `queue`: 557,8 -> 631,1 sfere (+13 per cento)	Invio asincrono (`vkQueueSubmit`)

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Coda dei job (2 job)	Fatta	Due blocchi di memoria job; l'ARM riempie uno mentre la GPU disegna l'altro.	`herotris` : 49956 triangoli a 13,7 ms (Q) e 50024 a 13,5 ms (Q2)	Frame in volo
Frame trattenuto un frame (`gpu3d_later`)	Fatta, spenta	Costa un frame di latenza.	+0,4 per cento `herotris` , +10 per cento `heroes`	-
Fusione dei draw (`gpu3d_merge`)	Fatta	Molti pezzi piccoli diventano una mesh in assi mondo. Solo da 32 pezzi.	100 pezzi: da +24 a +33 per cento; 10 pezzi grandi: -26 per cento	Instancing / batching dei draw
Texture RGBA8888 (formato T)	Fatta	Fino a 8 texture in un job.	`quad_tex` : 131,1 quarti di schermo	`glTexImage2D`
Texture RGB565 e RGBA5551	Fatta, spenta	Metà memoria e metà letture di texture (`gpu3d_tex16`). Nessun guadagno sul Pi (21 ott).	-	Formati compressi o impacchettati
Texture ETC1	Fatta, spenta	`gpu3d_etc1` , encoder in `src/gpu/etc1.c` . Nessun guadagno sul Pi.	-	ETC1 (estensione GL ES 2)
Filtro bilineare	Fatta	`gpu3d_filter=1` . L'aspetto differisce dall'ARM. Il 2D resta al texel più vicino.	Riempimento: 2,3 ns a pixel contro 1,65 del più vicino	`GL_LINEAR`
Attributi compatti dei vertici	Fatta, spenta	`gpu3d_attr=1` . `herotris` 54056 normale, 54013 compatto: nessun guadagno, il frame è limitato dall'ARM.	72,2 eroi contro 66,2 solo in `heroes`	Formati di vertice impacchettati
Ordinamento hardware (`gpu3d_sort`)	Scartata	Spenta: i numeri non mostrano guadagno.	`heroes` : 14,2 ms contro 15,9 ms ma il campo pixel era 0	Ordinamento della geometria trasparente
Cel shading sulle QPU	Fatta	Luce in 4 fasce nei programmi dei vertici.	Da x0,99 a x1,02 della luce del sole; 50 eroi a 12,1 ms	Toon shader
Contorno sulla GPU	Fatta	Guscio invertito nei programmi dei vertici.	Da x1,05 a x1,23 dell'eroe illuminato; 42,4 eroi a 14,4 ms	-

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Draw call	Fatta	Un cubo per chiamata.	1386 chiamate a 60 fps (12 us a chiamata; ARM 467)	Numero di draw call
Molti triangoli a 60 fps	Fatta	`herotris` con vertex shader e coda.	Circa 50000 triangoli a 13,7 ms	-
Calcolo sulle QPU	Fatta	Programmi fuori dal 3D: fade a livelli e passo di particelle (`gpu3d_qpu`). Li usa solo `dark_end` .	Fade da 8,6 a 9,3 ms contro 20,4 ms; particelle 1024 in 47-52 us contro 158-171 us	Kernel OpenCL; compute shader
Job concatenati (CT0/CT1)	Non fatta	Solo se una misura mostra la GPU ferma tra i job. Non lo è: il frame è limitato dall'ARM.	-	-
Geometry e tessellation shader	Non fatta	Il VideoCore IV non ha questi stadi.	-	Geometry e tessellation GL 4
Ray tracing	Non fatta	Nessun hardware; mai considerata.	-	`VK_KHR_ray_tracing`

3D sul Mali-G52 (RGB30)

Il Mali è un renderer a tile della famiglia Bifrost. bm costruisce i descrittori dei job da Mesa (`v7.xml` , MIT) e gli shader con il compilatore Bifrost di Mesa (`tools/mali/mkshader.c`). Fonte: `src/rgb30/mali.c` , `mali_desc.c` . Tabella di parità con il V3D: `docs/progress/mali-parity.md` .

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Triangolo con job Vertex, Tiler e Fragment	Fatta	Verificata sul dispositivo con un controllo dei pixel.	-	Catena di job Panfrost
Vertex shader (`BM_VS` , `BM_VS3`)	Fatta	Posizione in pixel, oppure una matrice 4x4 passata a ogni draw (vertici nello spazio del modello).	`draws` : 533,8 a 60 fps con lo shader contro 116,5 senza	Vertex shader
Fragment shader, colore piatto	Fatta	Un colore a draw, passato allo shader.	`quad_flat` : 143,2 quarti di schermo (ARM 15,8)	Fragment shader
Luce per faccia, cel e nebbia	Fatta	Calcolate dall'ARM in `be_mesh` , poi inviate come draw.	Il costo è sull'ARM: 6960 istruzioni a triangolo in `spheres` , contro 1213 in `spheres_unlit`	Luce per vertice
Texture (opache, più vicino)	Fatta	Descrittori Texture, Surface e Sampler, un pool da 2 MB, chiave `mali_tex` . Alpha, screen-door e bilineare restano sull'ARM.	`quad_tex` 6,4 -> 126,3; punteggio giochi 482 -> 530	`glTexImage2D`
Batching dei draw in un solo passo	Fatta	Arena da 4 MB, 1024 draw in un passo GPU; oltre si fonde nello spazio schermo.	Punteggio giochi 192 -> 247; punteggio GPU 14 -> 228 (bm3d 7.15)	Command buffer
Coda asincrona dei job (2 set)	Fatta	L'ARM lavora mentre la GPU disegna (`gpu3d_queue=1` o `2`).	2 frame in volo finiscono in 204 us	Frame in volo
Resolve nella pagina	Fatta	Resolve da 2 MB con cache; il colore chiave `0xFF00FF` segna "non coperto".	`resolve_us` da 0,8 a 2,1 ms in un frame da 14 ms	Present / blit

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Costruzione dei draw sui core	Fatta	Vedi la tabella dei quattro core sopra.	Giochi 451 -> 491	-
Z-buffer hardware e early z	Parziale	In lavorazione (traccia G2 di `docs/progress/multicore.md`). Oggi l'ARM ordina i triangoli dal lontano al vicino (`sort_tris`).	-	Allegato depth
Luce per faccia nel vertex shader	Non fatta	Voce Q2 della coda. Il Pi la fa nel suo vertex shader; il Mali solo trasforma.	`spheres` del Pi 782,9 contro 22,9	Illuminazione per vertice
Alpha blend e bilineare sulla GPU	Non fatta	Restano sull'ARM.	-	`glBlendFunc`, `GL_LINEAR`
MSAA	Non fatta	Nessuna impostazione Mali è usata.	-	MSAA
2D sul Mali	Scartata per gli sprite	Ogni draw costa circa 4 us, quindi 600 sprite costano 2,4 ms, più del percorso ARM (circa 0,5 ms). Conviene solo per layer grandi e fade, dopo il lavoro sulle texture.	-	-
Il Mali batte il raster ARM	Parziale	In alcune scene l'ARM su quattro core è avanti: `spheres_unlit` ARM 164,4 contro GPU+VS 103,8; lo stesso per `heroes`, `baked`, `shine` e `split`. Il Mali vince in `quad_flat` e `draws`.	Vedi le righe sopra	-

2D sulla CPU

Il codice 2D (`src/bm/gfx16.c`) è lo stesso su Pi e RGB30. I pixel sono RGB565.

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Primitive	Fatta	`line` , `rect` , `rectfill` , `circ` , `circfill` , `tri` (piatto o Gouraud con tre colori), `pset` , `pget` .	Non misurato da solo	`SDL_RenderLine` , `SDL_RenderFillRect` , OpenVG `vgDrawPath`
Rettangolo di clip	Fatta	`clip` limita ogni chiamata.	-	`SDL_SetRenderClipRect` , scissor
Sprite	Fatta	`spr` , `sspr` con zoom e flip; alpha a 1 bit (un pixel è acceso o spento).	2574 sprite 16x16 a 60 fps (6,5 us ciascuno)	`SDL_RenderTexture`
Mappa di tile	Fatta	`map()` fino a 8 layer, maschera dei flag dei tile, copia compatta delle celle (128 byte a cella).	Un layer opaco a schermo intero 3,08 ms; 7,7 layer entrano a 60 fps. Pool compatto: 2,9 -> 2,0 ms su Yharnam	Tilemap di un game engine
Testo bitmap	Fatta	Font 6x12, 8x14, 8x16, 16x32, 24x48, scala intera.	Non misurato da solo	Uso bitmap di `SDL_ttf`
Sprite stacking, rotazione di una fetta	Fatta	`stack()` ruota e scala una fetta (campionamento al più vicino).	54,6 stack da 8 fette a 60 fps	Trasformazione affine di un'immagine (OpenVG)
Tabelle di luce (`fades` , `g16_fade_apply` , `g16_fade_glow`)	Fatta	Buio e lampade con una tabella di livelli di colore.	Fade a schermo intero 20,3 ms; luce 30,7 ms (Pi); a 30 fps 24,7 e 20,3 lampade	Tabella di lookup dei colori
Modi schermo	Fatta	`screen(w, h)` da 320x180 a 1920x1080; lo scaler HDMI ingrandisce senza costo CPU.	-	`SDL_SetRenderLogicalPresentation`

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Blit bitmap con scala	Parziale	Funziona. Divide per ogni pixel; l'ARM1176 non ha l'istruzione di divisione. Una tabella di indici x sarebbe da 2 a 3 volte più veloce (stima).	Non misurato	`SDL_RenderTexture` con scala
Disegno del testo a run	Parziale	`g16_text` controlla ogni pixel. Disegnare run sarebbe circa 2 volte più veloce (stima).	Non misurato	-
Bande su quattro core per blit, luce e `cls`	Non fatta	Solo RGB30, circa 3 volte sui passaggi a schermo intero (stima). Il modello a bande di `r3d` esiste.	-	-
`-O3` per file in 2D e 3D (`PI_O3_FILES`)	Parziale	Costruito e provato sull'host; non misurato sul Pi. Si accetta solo con il 3 per cento di `ms` in meno in `gpu2d` e `mix` e il 5 per cento di kernel in più.	-	-
Benchmark 2D	Fatta	`b3d tests=map8,sprites,stack,fade,light,gpu2d`.	Vedi `docs/PERF-BUDGET.md`	-
Rotazione sprite con pila di trasformazioni	Non fatta	Lacuna dell'API rispetto a LÖVE (API-7). Prima serve una nota di progetto.	-	`SDL_RenderGeometry`; trasformazione Vulkan
Render target fuori schermo	Non fatta	Lacuna dell'API (da API-8 a API-11).	-	`SDL_SetRenderTarget`; OpenVG `VGImage`
`palt` con alpha, modi di blend	Non fatta	Lacuna dell'API. L'unico blend è `g16_fade_glow`.	-	`SDL_SetTextureBlendMode`; modi di blend OpenVG

2D sulle GPU

Tecnica	Stato	Cosa fa	Misurato	Equivalente
2D sopra il 3D nel job V3D	Fatta	Rettangoli, punti, linee, `spr`, `sspr` (zoom interi), celle di `map` e testo di `print` diventano quad nello stesso job.	3569 sprite o più a 60 fps (ARM 2574)	Quad con texture
Luce dalla GPU (`gpu3d_light`)	Fatta	Il V3D illumina la pagina che `dark_begin` .. `dark_end` costruisce. Fino a 12 lampade in un tile di 64x64 pixel. Le lampade si sommano; la luce ARM ha livelli.	Yharnam: luce 12,9 ms -> 1,7 ms; 31,6 -> 50,1 fps in autoplay. Solo Pi	Passo di luce deferred
Fade dalla QPU (`gpu3d_qpu`)	Fatta, spenta	Fade a schermo intero sulla QPU.	Da 8,6 a 9,3 ms contro 20,3 ms	Passo di post-processing
Menu sul V3D (`menu_scale`)	Fatta	Il menu è disegnato a 720p o 1080p dal V3D, costo di ingrandimento 0. 1080p verificato; 720p non ancora.	-	-
Mappa come batch residente di quad	Non fatta	L'idea: un blocco di 16x16 celle è un buffer fatto una volta. Il 2D GPU di oggi ricostruisce ogni quad: più lento dell'ARM su una mappa grande (44,3 fps contro 50,5 su Yharnam).	Stima: da 2 a 2,5 ms di guadagno, non misurato	Vertex buffer object
Solo 2D sulla GPU (`gpu2d_only`)	Scartata per le mappe grandi	Più lenta dell'ARM.	Vedi sopra	-
Luce dal Mali	Non fatta	Nessun equivalente Mali dello shader di luce del V3D. Si considera solo se Yharnam gira sotto 60 fps sulla RGB30.	-	-

Tecnica	Stato	Cosa fa	Misurato	Equivalente
Fade di `dark_end` come fragment shader	Non fatta	Le tabelle di colore hanno 65536 voci. Entra solo una scala senza tabella, e le immagini sarebbero diverse.	-	-
Copia nella pagina dalla GPU	Scartata	Il DMA richiede 3,1 ms contro 1,8 ms dell'ARM.	-	-

L'elenco OpenVG e VGU (vettoriale 2D)

bm non adotta l'API OpenVG: la sua versione per il firmware del Pi richiede i servizi del firmware VideoCore, e il Mali non ha OpenVG. Ogni voce sotto sarebbe una funzione di bm, nello stile di `docs/api/draw.md`. I costi sono stime dal codice. Fonte: `docs/progress/graphics-vector.md`. L'antialiasing è fuori ambito.

Tecnica	Stato	Cosa fa	Stima	Equivalente
Vernice a colore pieno	Fatta	Ogni chiamata di disegno.	-	`vgSetColor`
Gradiente lineare	Parziale	`tri` con tre colori; `rectgrad` sarebbero due triangoli Gouraud.	0,05 ms	Gradiente con `vgSetParameter`
Gradiente radiale e spread	Non fatta	Differenze in avanti sulla distanza al quadrato e una tabella da 256 voci. Priorità 1.	Circa 6 cicli a pixel	Gradiente radiale OpenVG
Percorso (move, line, quad, cubic, arc, close)	Non fatta	Suddividere le curve, lista dei bordi e riempimento a scanline. Priorità 2.	3 ms a schermo intero; 0,02 ms per un'icona 32x32 sul Pi	`vgAppendPathData`
Regole di riempimento (even-odd, non-zero)	Non fatta	Parte del codice dei percorsi.	Circa 0 in più	`VG_EVEN_ODD`
Tratto (larghezza, estremi, giunzioni, miter, tratteggio)	Non fatta	Quad spostati e giunzioni tonde. Esistono solo linee da 1 pixel.	-	`vgDrawPath(VG_STROKE_PATH)`
Forme VGU (rettangolo tondo, ellisse, arco, poligono)	Non fatta	`rrect`, `ellipse`, `poly` sopra `circfill` e `rectfill`. Priorità 1.	`rrect` 0,03 ms	`vguRoundRect`, `vguEllipse`
Trasformazione affine di un'immagine	Parziale	`stack` ruota. `sspr_affine` con una matrice 2x2 userebbe `stack_core`. Priorità 1.	16x16: 0,01 ms (Pi)	`vgDrawImage` con matrice
Deformazione prospettica (VGU)	Non fatta	Due triangoli con texture. Priorità 3.	0,02 ms per 16x16	`vguComputeWarpQuadToQuad`

Tecnica	Stato	Cosa fa	Stima	Equivalente
Formati immagine con alpha vero	Parziale	Oggi l'alpha è acceso o spento. Un byte di copertura 2-255 nello sheet funzionerebbe con `blit_alpha` .	Circa 8 cicli a pixel in più dove l'alpha non è binario	Immagini ARGB
Modi di blend	Non fatta	`blend(mode)` : alpha, add, multiply, screen.	Schermo intero 2 ms (Pi); 0,5 ms con le bande (RGB30)	`VG_BLEND_*` ; `SDL_BlendMode`
Layer maschera	Parziale	Solo i flag dei tile.	Un caricamento in più a pixel	`vgMask`
Vernice a motivo	Parziale	`spr` e `map` coprono la maggior parte degli usi.	-	Pattern paint
Matrice di colore, convoluzione, sfocatura gaussiana	Non fatta	Sfocatura box con somme correnti. Sfocatura 320x180: 1,2 ms (Pi); 0,1 ms con le bande (RGB30).	Vedi il testo	`vgColorMatrix` , `vgGaussianBlur` ; filtri OpenVX
Font vettoriali	Non fatta	`stb_truetype` (pubblico dominio), glifi in cache come sheet alpha.	Da 0,05 a 0,1 ms a glifo; blit in cache 0,01 ms	`vgDrawGlyph` ; `SDL_ttf`
Antialiasing delle forme 2D	Scartata	Fuori ambito (utente, 28 ott).	-	`VG_RENDERING_QUALITY`

Ordine di lavoro per il 2D, concordato con l'utente: prima le migliorie al codice esistente, poi `rectgrad` , `rrect` , `ellipse` ; poi `sspr_affine` e alpha; poi percorsi e tratto; poi font vettoriali e sfocatura. Il Mali prende il 2D solo per layer grandi.

Cosa hanno le altre API e bm no

API	Funzione	bm
OpenGL ES 2 / Mesa vc4	Shader programmabili sul VideoCore IV	Fatta, con assemblatore QPU proprio (`tools/qpuasm.py`), senza OpenGL
Vulkan	Command buffer, sincronizzazione esplicita, più code	Parziale: una coda di job con due set, nessuna API generale di comandi
Direct3D	Feature level, shader model	Non applicabile
OpenCL	Calcolo generale	Parziale: programmi QPU per il fade e le particelle. Niente OpenCL
OpenVX	Grafo di filtri immagine (sfocatura, convoluzione, colore)	Non fatta (vedi l'elenco OpenVG)
SDL2 / SDL3	`SDL_Renderer` : modi di blend, scala, render target, geometria	Parziale: scala e clip. Niente modi di blend, niente render target
Mesa Panfrost	Driver aperto per Bifrost	Usato: bm prende i descrittori dei job (`v7.xml`) e il compilatore degli shader
Mesa vc4 / v3d	Driver aperto per VideoCore	Letto per le strutture e i probe. Il codice Linux si legge, non si copia

Punti aperti

- Pi Zero W: le scene 2D `map8`, `sprites`, `stack`, `fade` e `light` sono state misurate una volta (22 ott). La lista `-O3` per `gfx16.c` non è misurata.
- RGB30 con il Mali: esistono solo le cifre del 3D Bench del 25 e 28 ott. Le scene 2D non sono misurate sulla RGB30.
- Flip e alpha da soli, particelle 2D e particelle 3D sul Pi: non misurati.
- **[DA COMPLETARE: le cifre dello Z-buffer hardware del Mali-G52, a fine traccia G2]**
- **[DA COMPLETARE: le cifre RGB30 delle bande 2D su quattro core, che non sono costruite]**

Punti aperti

- **Punti aperti**: le cifre dello Z-buffer hardware del Mali-G52, a fine traccia G2
- **Punti aperti**: le cifre RGB30 delle bande 2D su quattro core, che non sono costruite